



ROBOCUPJUNIOR RESCUE MAZE 2025

TEAM DESCRIPTION PAPER

RoboDanesh Z



Abstract

This paper presents the design and development of an autonomous rescue robot by Team RoboDanesh for the RoboCup Rescue Maze competition. The robot is designed to operate reliably in complex and dynamic environments, simulating the challenges of real-world disaster scenarios. Its main capabilities include autonomous navigation, path planning, obstacle and wall detection, ramp traversal, victim identification, and rescue kit deployment.

The robot is built on an integrated mechanical, electronic, and software architecture. The mechanical design emphasizes stability, durability, and modularity, allowing reliable movement over uneven surfaces. The electronics are centered around a main controller supported by custom circuit boards, enabling efficient sensor integration, motor control, and communication. Environmental perception is achieved using **VL53L1X** distance sensors for walls and obstacles, Microswitches for physical contact, **APDS-9960** and **IR sensors** for surface color recognition, and a **CMPS14** digital compass for orientation. The software architecture combines structured navigation and decision-making algorithms with vision-based victim detection. Image processing is handled on a **Raspberry Pi 5** using Python and established computer vision libraries. Particular focus was placed on integrating subsystems, validating performance through testing, and refining the robot iteratively. These design choices result in a stable, responsive, and adaptable robot capable of reliable performance in challenging rescue maze scenarios.

1. Introduction

a. Team

- **Arman Jamshidi (Control):** Competing since 2021 in Rescue Maze and Simulation leagues, he is responsible for the robot's control systems and navigation algorithms.
- **Arya Torabi (Image Processing):** Competing since 2021 in Rescue Line, Maze, and Simulation leagues, he is responsible for image processing, machine learning, and detection algorithms.
- **Sina FamilSabzevari (Mechanics & Electronics):** Competing since 2021 in Rescue Maze and Line leagues, He is responsible for mechanical design, assembly, and optimization. And also, in charge of designing and soldering the electrical circuits.
- **Rayan Saeid Ershadi (Control):** Competing since 2023 in Rescue Maze and Simulation leagues, he is responsible for the robot's movement systems and navigation algorithms.



Arman
Jamshidi

Arya
Torabi

Sina
FamilSabzevari

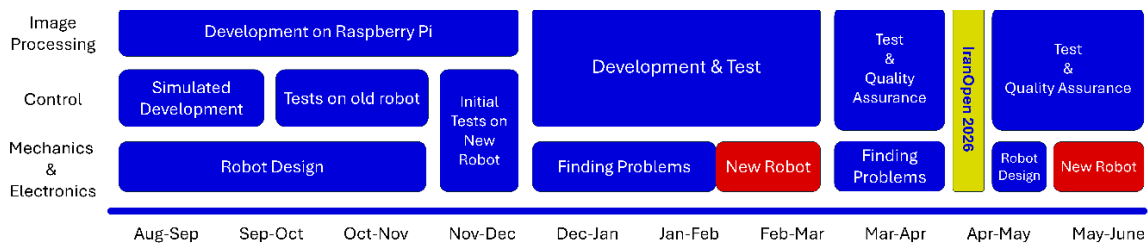
Rayan
Saeid Ershadi

2. Project Planning

a. Overall Project Plan

The project was developed in parallel across mechanical, electronic, and software parts. Mechanical and electronic work progressed simultaneously, while software was tested either on an older robot or in simulation.

After building the first full prototype, we tested it under real conditions, including full-load operation, suspension behavior, and system integration. These tests revealed issues that were fixed in later versions, improving overall performance and reliability. The process also helped the team improve teamwork, practical problem-solving, and develop new technical skills throughout the project.



b. Integration Plan

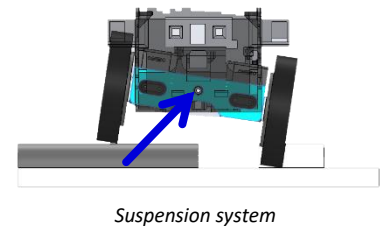
The robot was designed as a modular system, with each part built separately and then combined step by step. Each subsystem was developed to meet competition needs and to fix weaknesses from previous years. We used clear power and data connections between modules to make integration more reliable. After each integration step, the full system was tested to check stability and communication between parts and to confirm improvements over earlier versions.

3. Hardware

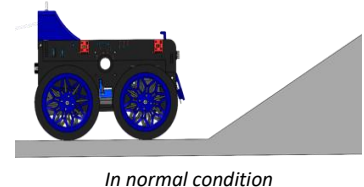
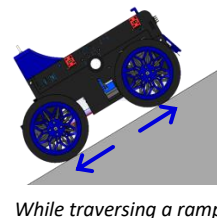
The robot is designed using the SolidWorks application. Each component is designed individually as a part (.sldprt) and then assembled into an assembly file (.sldasm). All custom parts are manufactured using 3D printing with PLA material and the Fused Deposition Modeling method. Since PLA can be fragile, the robot chassis is designed with a minimum thickness of 3 mm to reduce the risk of breaking. The chassis dimensions are 195mm×105mm with a height of 95 mms.

a. Mechanical Design and Manufacturing

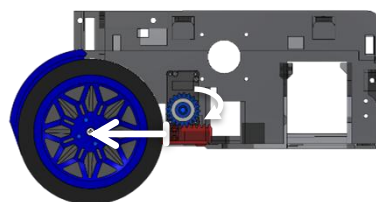
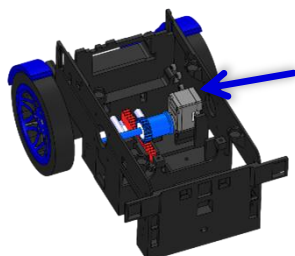
One of the important advantages of the robot is the front-wheel suspension system, which allows all four wheels to remain in contact with the ground while passing over uneven surfaces. To avoid collisions between the wheels, PCBs, and chassis, the suspension rotation is mechanically limited, as shown.



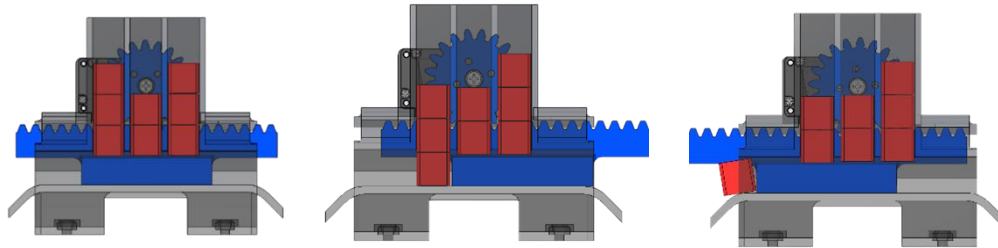
A major challenge was maintaining balance and smooth motion when traversing ramps, especially those with speed bumps. The initial solution was to increase the distance between the wheel's axles. This was achieved by moving the rear motor holder back and forth using a rack and pinion mechanism.



In normal conditions and while traversing a ramp, the distance between the wheel axles increases from 112.5 mm to 150.5 mm. The challenge of using this mechanism was proper weight distribution so that the rack alone would not bear the full load, which could damage both the rack and pinion or waste motor force. To solve this, two poles were fixed to the chassis and inserted into the motor holder, and several ball bearings were attached to small bumps on the chassis. Also, since the distance between the motor's torque and the place where the force is applied, a ball-bearing and a custom part were used to avoid damaging the motor.



The rescue kit delivery system uses a Dynamixel motor with a rack and pinion and a custom-designed holder. It consists of three columns with the rack placed beneath all three to prevent kits from falling. The pinion moves the rack to create space for one kit to drop, then moves again to push and deliver the kit as shown.



The robot also includes a handle, two front microswitch shields, and a lid that is mounted using magnets.

Two types of motors are used: **Dynamixel XC-430** for movement, **Dynamixel XL-330** for rescue kit delivery, and wheel axles change.

Dynamixel XL-330

Stall Torque	0.52 [N.m] (at 5.0 [V], 1.47 [A])
No Load Speed	103 [rev/min] (at 5.0 [V])
Input Voltage	3.7 ~ 6.0 [V] (Recommended: 5.0 [V])
Feedback	Position, Velocity, Load, Realtime tick, Trajectory, Temperature, Input Voltage, etc.

Dynamixel XC-430

Stall Torque	1.6 [N.m] (at 12.0 [V], 1.4 [A])
No Load Speed	106 [rev/min] (at 12.0 [V])
Input Voltage	6.5 ~ 14.8 [V] (Recommended: 12.0 [V])
Feedback	Position, Velocity, Load, Realtime tick, Trajectory, Temperature, Input Voltage, etc.



XC-430



XL-330

○ Reliability Tests and Quality Assurance

After confirming the efficiency of the suspension system, it was designed and virtually tested in SolidWorks. During testing, vibration issues were observed, so the part was placed between two short walls to reduce vibration. For the adjustable wheel axis distance, a test component was designed and printed to evaluate motor force and rack-and-pinion strength. After final assembly, the mechanism was tested under various maze conditions with different ramp and speed bump placements to ensure reliability.

• Innovation:

The axle distance increase is an important innovation of ours, alongside the suspension system, which is another key feature in the robot that can be considered.

Also, our rescue kits are made heavier by adding weights inside them(midprint), which reduces the risk of the kits bouncing off the desired radius from the victim.

b. Electronic Design and Manufacturing

• General Electronics Architecture

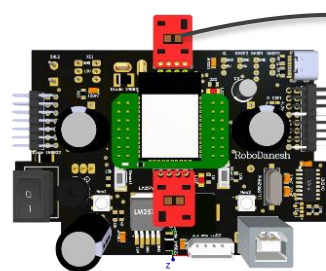
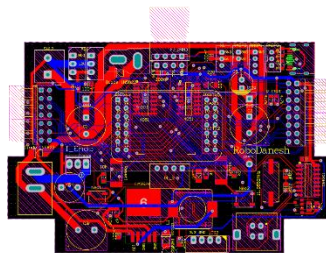
The robot electronics are based on a custom multi-board PCB system. The electronics are distributed across **5 purpose-built circuit boards + a regulator module**, making the system modular, easier to debug and repair, and dedicated to specific functions while still fully integrated as one unit.

All PCBs were designed using Altium Designer as double-layer boards and were hand-soldered, allowing precise control over layout and routing, which improved overall system reliability.

Note: In the board images, the GND polygon has been removed to make the PCB layout and routing more visible.

- **Main Board:** ESP32-WROOM, CH340 USB interface for program uploading, motor communication ports, 1 VL sensor, compass connector, and connection to the regulator board and side boards.

Overall Electronics Block Diagram

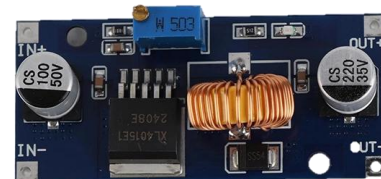


This VL is just placed there for Plan B, if we wanted to change the back VL placement.

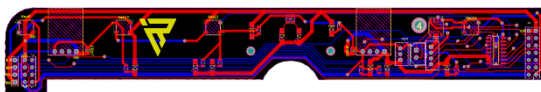
- **3.3V Regulator Board:** Provides a stable 3.3V supply for the ESP32 and sensors, ensuring reliable and consistent operation.



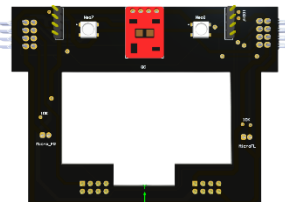
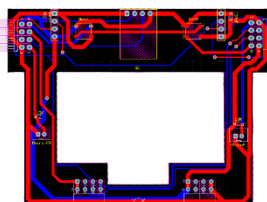
- **5V Regulator Module:** Provides regulated 5V power for components like the Raspberry Pi, IR sensor, and motors, keeping voltage stable under load.



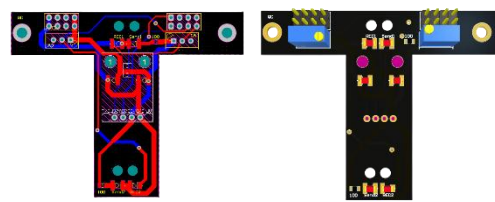
- **Side Boards (Left and Right):** 2 VL sensors, multiplexers, LEDs, and connection to main and front boards.



- **Front Board:** 3 VL sensors, LEDs, and connection to side and middle boards.



- **Middle Board:** IR sensors, APDS sensor, and connection to front board.



- **Main Controller:** The robot is controlled by an ESP32-WROOM microcontroller, chosen for its high performance and versatility. ATmega128 and ATmega2560 microcontrollers were considered due to team familiarity and Arduino support, but their low clock speed and limited memory made them unsuitable.

STM32 microcontrollers offered higher performance, but this was unnecessary. The ESP32 runs at up to 240 MHz, with 3.3 V logic and 34 GPIO pins, multiple communication interfaces, and low power consumption, making it ideal for sensors, motors, and communication. A CH340 USB-to-UART converter with USB-C and an AMS1117 regulator ensures stable programming and debugging.

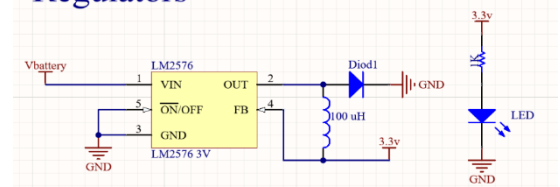
Microcontroller	ATmega128	Atmega2561	ESP32-WROOM	STM32
Flash Size	128 KB	256 KB	4 MB	256 KB – 2 MB
Memory Size	4 KB	8 KB	320 KB	4 KB – 1 MB
Clock Speed	16 MHz	Up to 16 MHz	Up to 240 MHz	16 – 550 MHz
GPIO Pins	53	86	34	16 – 168
Non-Volatile Storage	4 KB	4 KB	Emulated in Flash	Emulated in Flash
UART Count	2	4	3	1 – 8
I ² C Count	1	1	2	1 – 6

- Power System:** The robot is powered by a 12.6 V three-cell lithium-polymer battery. This voltage is converted to 5V and 3.3V using switching regulators mounted on the main board. The XL-4015 (5V) was chosen for its high current capability, stable output, and efficiency under heavy load, while the LM2576 (3.3V) was selected for its reliable and stable regulation for low-voltage control and sensor systems.

The 5V rail supplies the XL330 motors and Raspberry Pi 5, and the 3.3V rail supplies the ESP32 and sensors, such as VL53L1X, APDS9960, and CMPS14, IR sensors, and LEDs.

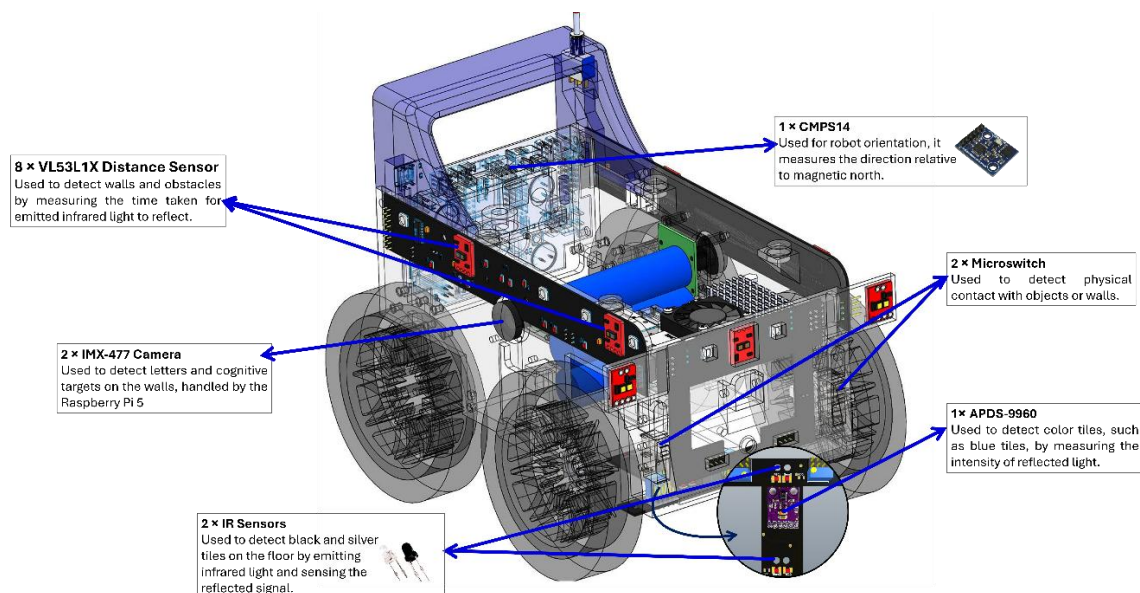
Using switching regulators and wide PCB traces helps keep the voltage steady, reduces heat, and allows the system to handle high currents reliably.

Regulators



- Sensors**

The robot uses **VL53L1X** distance sensors, **CMPS14** compass, **APDS-9960** color sensors, **IR** sensors, and **Microswitches** sensors to collect information about the environment.



Multiplexers are used to connect multiple I²C sensors, LEDs, and motors to the TWC bus. This makes it possible to switch between different devices using the same lines and reduces the number of ESP32 pins needed.

- **Communication Structure**

I²C Bus 1: VL53L1X Sensors and APDS-9960

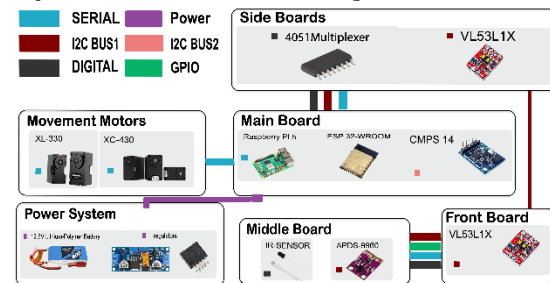
I²C Bus 2: CMPS14 Compass

UART: Raspberry Pi 5, Motors, USB uploading

DIGITAL: Microswitches

POWER: 12.6V Lithium-polymer battery

System Communication Diagram



- **Innovations**

The PCBs connect via pin headers, which eliminates the use of wires, which helps keep the connections organized and, more importantly, reduces unwanted noise.

- **Improvements**

In the previous design, the Raspberry Pi received power through multiple intermediate boards, causing a voltage drop under high load. In the updated version, it was connected directly to the main board for more stable power delivery. A dedicated connector board was also added to improve connection reliability and reduce the risk of loose wiring and short circuits.

The power system was also upgraded by replacing the **LM61460 5V 6A** regulator with the **XL4015 5V** regulator, while the **LM2576** remained in use for 3.3V devices. This improved voltage stability, efficiency, and overall reliability under full system load.

- **Reliability Tests and Quality Assurance**

The electronics were tested under full load with all sensors, motors, LEDs, and the Raspberry Pi operating simultaneously. At this condition, it was found that the 5V rail carries around 2.6A and the 3.3V rail carries about 0.6A. Voltage stability, current delivery, and communication between modules were checked during testing. The PCBs were also inspected after assembly for soldering and connection issues.

4. Software

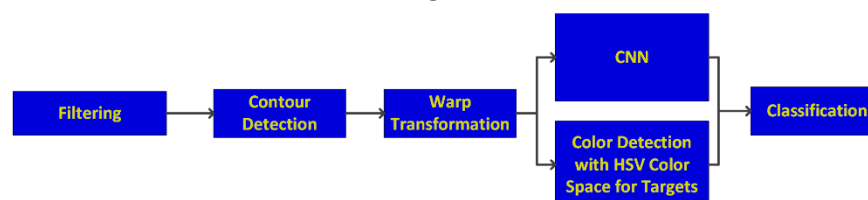
a. General software architecture

The software of the robot is separated into 2 parts: **Image Processing** and **Control**, where the Image Processing part handles the victims at the sides of the robot on the Raspberry Pi, and the Control part handles the decision-making and movement of the robot using ESP32. This structure divides the processing load between the two devices and causes the high processing power of the system.

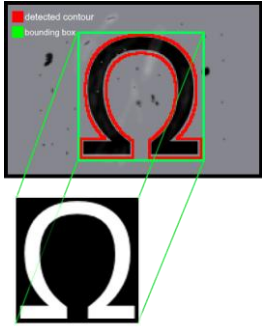
a. 1. Image Processing Subsystem Architecture (Raspberry Pi)

- **Overall Code Structure and Code Flow**

The victim detection code is written in Python and runs on the Raspberry Pi using NumPy, OpenCV, and Picamera2. The image is processed through a sequential pipeline including **Filtering**, **Contour Detection**, **Warping**, and **Classification** to identify valid victims. After detection, the final result is sent to the ESP32 via the Raspberry Pi's serial interface for further handling.

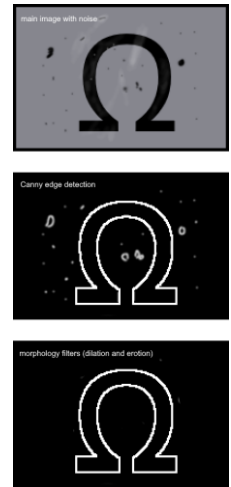


• Preprocessing (Cropping, Warping, Filtering)



The first step of the object detection model is detecting the Edges of the frame using [OpenCV's Canny\(\)](#) method, which is then enhanced by applying a set of [morphology filters](#) (e.g., dilate and erode) to remove extra noise. The next step is Contour detection to get the necessary properties of the letter/target, e.g., bounding box, minimum area rectangle, aspect ratio, etc. (see [Contour Properties](#)). These features are analyzed to remove most of the unnecessary contours (known as false detected contours).

For the cognitive targets, an ellipse is fit to the contour (using OpenCV's "fitEllipse" function), then the area of the contour is compared to the area of the ellipse; if the areas are sufficiently similar, the contour is a candidate for the target detection model. The last step of the model is using OpenCV's "warpPerspective" function (as shown in the image) to separate the letter/target into a new image, and another thresholding step is applied to prepare the letter for the classification model.



```

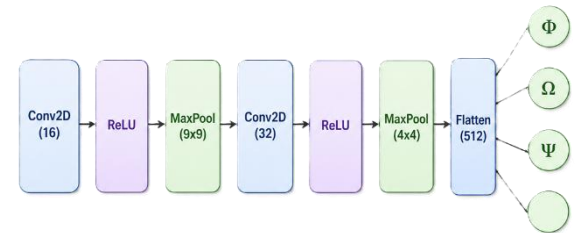
For each contour
  box = boundingBox(cnt)
  rect = minAreaRect(cnt)
  solidity = (cnt_area)/(box.area)
  aspect_ratio = rect.width/rect.height
  ...
endfor

If cnt_area > 100 and
solidity > 0.25 and
aspect_ratio > 0.4 and
...
Then #next step
newframe = warpPerspective(frame)
otsuThreshold(newframe)

```

• Letter Recognition

For the letter recognition task (Φ , Ω , Ψ), a **Convolutional Neural Network (CNN)** was used due to its effectiveness in image classification. Convolutional layers extract visual features, followed by **ReLU activation** and **MaxPooling** to reduce dimensionality. The extracted features are then passed to fully connected layers for classification. To prevent overfitting, **Dropout** is applied during training. The output layer uses **Softmax** to produce a class.



probabilities, while **Cross-Entropy Loss** and the **Adam Optimizer** are used for training. The diagram illustrates the exact architecture and layers of the model, while the pseudo-code shows the main training loop used for model training.

```

Select device(CPU)
Initialize CNN model
Define CrossEntropy loss and Adam Optimizer

For each epoch
  Train phase start
    For each batch in train_loader
      Move data to device
      Forward pass and compute loss
      Backpropagate and update
    endfor
  Phase end
endfor

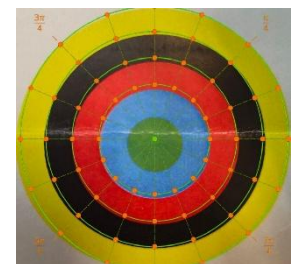
Evaluation phase start
Disable gradients
For each batch in test_loader
  Forward pass
  Compute loss and accuracy
endfor

If test accuracy improves
  Save model
Else
  Skip model

```

• Cognitive Target Detection

A polar sampling algorithm was implemented to detect circular targets. The target is divided into five radial segments, with 16 sampling points per segment. Pixel colors are analyzed in the **HSV** color space, and each segment's final color is determined using majority voting to reduce noise. The result of this processing can be seen in the image shown beside.



a. 2. Control Subsystem Architecture (ESP32)

• Overall Code Structure and Code Flow

The control system is written in C++ and runs on the ESP32. It is built around three main parts: Mapping, Decision-Making, and Movement. Other software components are used to provide the information and support required by these core parts, including sensor processing, Raspberry Pi communication, etc.



• Task-Based Design Using FreeRTOS

The ESP32 control subsystem is built on FreeRTOS and uses multiple tasks across both processor cores. The main task handles navigation, mapping, and decision-making, while the remaining tasks manage communication, motor control, and sensor processing. This structure has been chosen to improve responsiveness, minimize latency, and enable reliable real-time robot operation.

• Mapping and Environment Check

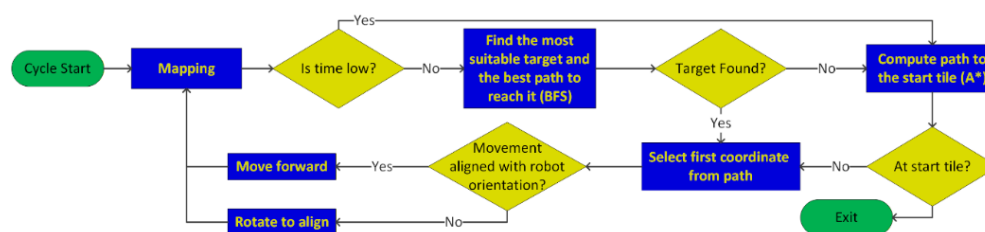
To support decision-making, the robot maps the environment, storing information such as walls, tile colors, and field features before each movement. A 3D grid map is built as an array, where each tile is indexed by its (x, y, z) coordinates. Environmental data for each tile is stored as bits within a structured data type, allowing compact memory usage and fast coordinate-based access. Walls are detected using VL53L1X distance measurements, tile colors are identified using IR and APDS9960 sensors, while ramps or stairs are detected using the CMPS14 sensor. At intersections, unvisited paths are saved as target tiles, enabling the system to distinguish between explored and pending areas.

If the robot moves beyond the boundaries of the predefined 3D map, the corresponding tiles are stored in a secondary coordinate-based map. These tiles are accessed through coordinate searches, allowing the map to expand beyond the fixed array size without increasing its memory footprint.

• Decision-Making Cycle

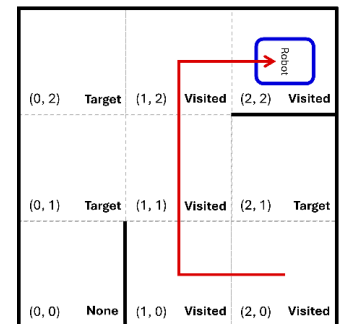
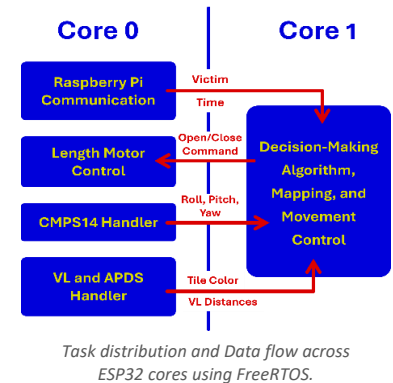
The robot's decision-making is based on a repetitive cycle executed before each movement. Path planning is handled using a hybrid approach: The A* algorithm is used to compute the optimal path to a specific target (such as the start tile), while the BFS algorithm is used to select the most suitable target during exploration. Additional costs, including turn costs and blue tile costs, are integrated into the path planning to account for directional changes and time delays on special tiles. After each movement, the decision-making cycle restarts, allowing frequent replanning to handle map inaccuracies, sensor noise, dynamic obstacles, and newly discovered paths.

When the remaining time (received from the Raspberry Pi) becomes shorter than the estimated return time, the start tile is set as the main target.

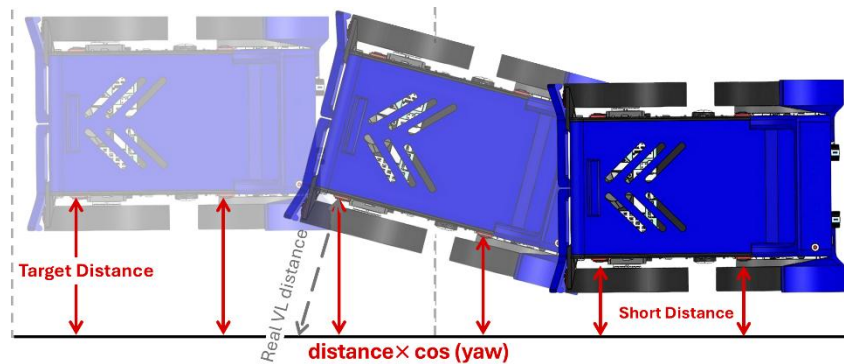


• Motion Control and Movement

The robot estimates its displacement using motor encoder feedback, enhanced by pitch and angular corrections to maintain accuracy on uneven terrain. A PD-based controller continuously corrects heading and



alignment using VL53L1X distance sensors and the CMPS14 compass, ensuring smooth and accurate motion with minimal oscillation during straight paths. Microswitches are used to detect physical obstacles, and detected obstacles are stored in the map and incorporated as additional costs in the path-planning algorithm.

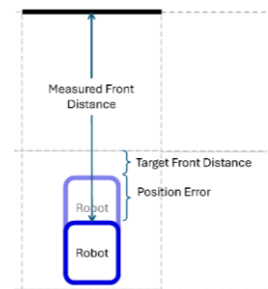


• Sensor-Based Motion Correction

The robot uses a two-layer sensor-based motion correction system. A PD controller with distance and compass data maintains stable tile-to-tile motion. Additionally, front and rear VL53L1X sensors correct forward/backward offsets by adjusting straight movements or re-centering before rotations, ensuring accurate alignment and reliable navigation.

Compass Yaw Calibration

After some time, the compass yaw starts to drift and develop an offset. To fix this, the robot aligns itself with a front wall using two microswitches, then measures the difference between the current yaw and the expected heading (0° , 90° , 180° , or 270°), using it as a correction for future readings.

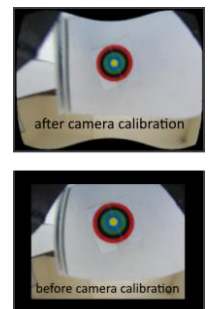


b. Innovative solutions

• Camera Distortion Fix

Since the IMX477 cameras have a wide lens angle, to fix the distorted corners of the frame, a [camera calibration](#) matrix was calculated (using the FOV of the camera and a few variables achieved with trial and error), which reverses the wide lens effect, improving detection of victims captured in the corners of the images.

To get the best parameters for each filter or range, OpenCV's trackbar feature was used to test the quality of the filter in real time, along with other OpenCV features to print more information on the frame.



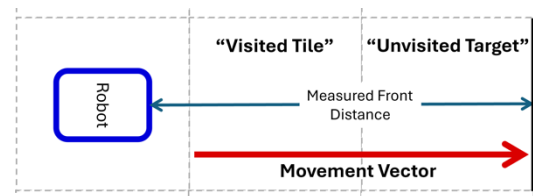
• Letter Detection Algorithms

Several creative design choices were implemented to improve performance and robustness. Instead of using a pre-existing dataset, all images used for training were collected by our team under different lighting conditions, distances, and viewing angles to better reflect real competition scenarios. The use of polar sampling instead of full-image analysis also reduces computation while improving stability in detecting circular targets. Additionally, custom HSV thresholds were tuned specifically for the environment to achieve more reliable color detection.



• Optimized Continuous Tile Traversal

To reduce stop-and-go motion, the robot optimizes forward traversal over consecutive tiles. Using the planned path and stored map in addition to distance sensor validation, it identifies multiple unvisited tiles and moves through them continuously, improving efficiency while maintaining safe navigation. The RTOS structure helps this movement a lot by minimizing the latency for reading sensor data between tiles.



• Ramp Mapping

Instead of converting height measurements into discrete z-levels, the robot stores the actual measured height for each floor. This minimizes the influence of ramp geometry and localization errors on the mapping process.

a. 3. Reliability Tests and Quality Assurance

Reliability was confirmed through extensive tests identifying weaknesses and bugs.

- **Control:** Before the tests on the real robot, pathfinding, mapping, movement, and motion corrections were tested on ESP32 and in Webots simulations with diverse layouts and obstacles. Artificial sensor noise was added, guiding improvements like median filtering for more accurate navigation and motion stability.
- **Quality Assurance:** Quality assurance involved testing in diverse fields and lighting, focusing on high-speed victim detection and stable obstacle traversal. Results were used to refine algorithms and enhance overall reliability and performance.
- **Quality Assurance – Image Processing:** Each run is saved with both camera feeds and detailed frame annotations. These can be manually labeled and compared with the model's output to evaluate detection accuracy and quickly detect errors or improvements in the algorithm. ([See here for real tests](#))

The letter detection algorithm system was evaluated under different lighting conditions and target positions to ensure robustness. Techniques such as dropout and majority voting improve stability and reliability, enabling consistent performance in competition environments.

Additionally, training and testing loss curves are provided, illustrating the model's learning behavior during training and showing the convergence and generalization performance of the network.

5. Performance evaluation

The robot was tested in maze environments similar to RoboCupJunior Rescue Maze conditions, including ramps, intersections, narrow paths, obstacles, and different floor colors. The evaluation focused on autonomous navigation, wall detection, victim identification, and rescue kit delivery.

Testing was performed iteratively. Sensors, motors, and communication systems were first tested individually, followed by full-system tests with all subsystems active. Multiple trial runs were used to evaluate navigation accuracy, sensor reliability, and overall robot stability, while voltage and current levels were monitored during operation.

Based on the test results, adjustments were made to control parameters, sensor placement, and software logic to improve reliability and overall performance.

6. Conclusion

Developing this robot required combining mechanical design, electronics, control, and image processing into a single working system. Throughout the project, different parts of the robot were repeatedly tested and improved to make the overall system more reliable during maze operation. Many design decisions, especially in sensor integration and modular construction, were based on problems observed in previous robots and during testing.

Beyond the technical side, the project helped the team improve teamwork, troubleshooting, and practical engineering skills. Working on the robot also gave team members experience with integrating different subsystems and solving problems under real competition conditions.

References

General

- robodanesh.com

Mechanics

- [SOLIDWORKS](#)
- ROBOTIS, *Dynamixel XC430 Series e-Manual*, 2024. Available at: [Dynamixel XC430 e-Manual](#)
- ROBOTIS, *Dynamixel XL330 Series e-Manual*, 2024. Available at: [Dynamixel XL330 e-Manual](#)
- SolidWorks Corporation, *SolidWorks CAD Software Documentation*, 2024. Available at: [SOLIDWORKS Documentation](#).
- ZHAOWEI Motor, *ZGA-12 DC Gear Motor Specifications*, 2024. Available at: [ZGA-12 Motor Specifications](#)
- Ultimaker, *PLA Material Properties and 3D Printing Guide*, 2023. Available at: [Ultimaker PLA Guide](#)

Electronics

- [Altium](#)
- [Texas Instruments](#)
- Texas Instruments, “LM61460 6-A Synchronous Step-Down Voltage Converter.” Available at: [LM61460 Product Page](#)
- Texas Instruments, “LM2576 SIMPLE SWITCHER® Step-Down Voltage Regulator.” Available at: [LM2576 Product Page](#)
- Espressif Systems, “ESP32-WROOM Series Datasheet.” Available at: [ESP32 Modules](#)
- Raspberry Pi Foundation, “Raspberry Pi Documentation.” Available at: [Raspberry Pi Documentation](#)
- STMicroelectronics, “VL53L1X Time-of-Flight Distance Sensor.” Available at: [VL53L1X Sensor Page](#)
- SparkFun Electronics, “APDS-9960 RGB and Gesture Sensor Hookup Guide.” Available at: [APDS-9960 Guide](#)
- Robot Electronics, “CMPS14 Tilt Compensated Compass.” Available at: [CMPS14 Compass](#)
- Altium, “Altium Designer Documentation.” Available at: [Altium Designer Documentation](#)

Software – Image Processing and Machine Learning

- [Raspberry Pi](#)
- [Python](#)
- [NumPy](#)
- [OpenCV](#)
- Hands on machine learning with scikit-learn and pytorch by Aurélien Géron
- [Stanford Cs229](#)
- Mathematics for machine learning by Marc peter Deisenroth ,A. Aldo Faisal ,Cheng soon ong
- Aryamehr(sharif) university of technology machine learning course by prof. Ali Sharifi Zarchi
- https://www.youtube.com/@Sharif_Machine_Learning

- <https://github.com/SharifiZarchi/Introduction to Machine Learning>
- Machine learning a probabilistic perspective by Kevin P. Murphy

Software – Control

- [Arduino](#)
- [Espressif Systems](#)
- [Espressif FreeRTOS](#)